3a. Create methods Add(), Multiply(), Subtract(), Divide() and call them using a C# Delegate
Solution:

```
using System;

namespace DelegateDemo
{
   // Delegate Declaration
   delegate void Calculator(int a, int b);

   class Program
   {
      static void Add(int a, int b)
      {
         Console.WriteLine("Addition = " + (a + b));
      }

      static void Subtract(int a, int b)
      {
         Console.WriteLine("Subtraction = " + (a - b));
      }

      static void Multiply(int a, int b)
      {
         Console.WriteLine("Multiplication = " + (a * b));
      }

      static void Divide(int a, int b)
      {
         if (b != 0)
            Console.WriteLine("Division = " + (a / b));
         else
            Console.WriteLine("Division by zero is not allowed.");
      }

      static void Main(string[] args)
      {
         Calculator c;

         c = Add;
         c(20, 10);

         c = Subtract;
         c(20, 10);

         c = Multiply;
         c(20, 10);

         c = Divide;
         c(20, 10);

         Console.ReadKey();
      }
   }
}
```

Output:

```
Addition = 30
Subtraction = 10
Multiplication = 200
Division = 2
```

3b. Write a program using Multicast Delegate

A **multicast delegate** is a delegate that can call **more than one method** using a single delegate object. It stores a list of methods instead of just one method. When the delegate is invoked, it executes all the methods in the order they were added. It is created by using the += operator to add methods to the delegate.

Solution:

```
using System;

namespace MulticastDelegateDemo
{
   delegate void ShowMessage();

   class Program
   {
      static void Welcome()
      {
         Console.WriteLine("Welcome");
      }

      static void Learn()
      {
         Console.WriteLine("Learning C# Delegates");
      }

      static void ThankYou()
      {
         Console.WriteLine("Thank You");
      }

      static void Main(string[] args)
      {
         ShowMessage msg;

         msg = Welcome;
         msg += Learn;
         msg += ThankYou;

         msg();

         Console.ReadKey();
      }
   }
}
```

Output:

```
Welcome
Learning C# Delegates
Thank You
```

3c. c. Create a class BankAccount with AccountNumber and Balance. Implement property for Balance with validation (Balance cannot be negative).

Solution:

```
using System;

namespace BankAccountDemo
{
   class BankAccount
   {
      public string AccountNumber { get; set; }

      private double balance;

      public double Balance
      {
         get
         {
            return balance;
         }
         set
         {
            if (value >= 0)
               balance = value;
            else
               Console.WriteLine("Balance cannot be negative.");
         }
      }
   }

   class Program
   {
      static void Main(string[] args)
      {
         BankAccount account = new BankAccount();

         account.AccountNumber = "123456789";
         account.Balance = 5000;

         Console.WriteLine("Account Number : " + account.AccountNumber);
         Console.WriteLine("Balance : " + account.Balance);

         account.Balance = -1000;

         Console.WriteLine("Balance : " + account.Balance);

         Console.ReadKey();
      }
   }
}
```

Output:

```
Account Number : 123456789
Balance : 5000
Balance cannot be negative.
Balance : 5000
```